

Natural Language Query for CCTV Search: Project Explained

A simple guide to this project for a minor-project proposal and viva.

Deeper references: [OVERVIEW.md](#) (full technical walkthrough) and [INTERVIEW_QNA.md](#) (54 detailed Q&As).

The project in one line

A security guard or CCTV operator types a question in plain English, like *"show me the stolen red bike with plate HR5653RT78 from last 7 days on the 192.168.9 cameras"*. The system turns that sentence into an exact search query that a video-surveillance system can run.

1. Problem statement

Modern CCTV / Video Management Systems run AI analytics such as number-plate recognition (ANPR), face recognition, crowd detection and helmet/safety-gear detection. They record thousands of events every day. To search these events, an operator has to fill a complex form: pick the analytics type, pick filters (colour, plate, speed), pick a date range and pick cameras from a list of hundreds. This is slow and needs training, and it breaks down in an emergency.

This project lets the operator type the search as a normal sentence. The system turns it into a structured, validated search request automatically.

What it solves

- **No more form-filling.** The operator writes one sentence instead of making 10+ dropdown selections.
- **It understands messy language:** typos in camera names, "last week" versus "past week", and several things asked in one sentence.
- **The existing backend doesn't have to change**, because the output is the exact JSON the search API already accepts (`transform.py`).

Objectives

1. Understand a free-text search query and identify which analytics (events) it refers to.
2. Extract filter conditions (colour, plate number, speed, emotion, etc.) for each event.
3. Convert time phrases ("last week", "yesterday 9am to 5pm") into exact date ranges.
4. Match camera names and IP addresses mentioned in the query, even with typos.
5. Produce a validated JSON payload for the backend search API and show the matching events.
6. Measure accuracy on a labelled test set.

2. How it works (with an example)

Input: *"stolen red bike, plate HR5653RT78, last 7 days, from 192.168.9"*

The query is split into 5 parts:

Part	Question it answers	Output
Events	Which analytics?	ANPR (number-plate recognition)
Fields	Which filters?	Category = Stolen, Colour = Red, Plate = HR5653RT78
Attributes	Person details?	(e.g. address = "MG Road")
Time	Which dates?	04-09-2026 00:00 to now
Cameras	Which cameras?	all cameras with IPs starting 192.168.9

Every result also records **which words in the query produced it** (source attribution). For example, "stolen red bike" produced Category = Stolen.

The flow ([graph.py](#))

```

User query
|
+--> Event selection (LLM)      --+
+--> Attribute extraction (LLM) --+  run in parallel
      |
      Relevant? -- no --> "irrelevant" (e.g. "what's the weather")
      | yes
+--> Time extraction (LLM classifies + Python calculates dates)
+--> Camera matching (pure Python fuzzy matching, no AI)
+--> Field extraction (LLM, one call per event)  all in parallel
      |
      Final JSON --> search the events --> show results

```

The key design idea

"The LLM does the language understanding, and plain code does anything that must be exactly right."

- The LLM decides *what kind* of time phrase "last week" is. Python does the actual date maths, because LLMs are bad at calendars.
- The LLM suggests event and field names. Code checks them against a whitelist and JSON Schemas, and drops anything the LLM made up (a "hallucination").
- Camera names are matched with **RapidFuzz** fuzzy string matching, with no AI involved, so a typo like "dr peper" still finds `DR_PEPPER_ENTRY_CAM01`.
- If any step fails (for example, the AI service is down), that step returns a safe default instead of crashing the app.

Modules

Module	Folder	What it does
Event selection	<code>nodes/events/</code>	Picks which of the 15 analytics types the query is about
Attribute extraction	<code>nodes/attributes/</code>	Finds person/entity details like address
Field extraction	<code>nodes/fields/</code>	Finds filter conditions for each event
Time extraction	<code>nodes/time/</code>	Classifies time phrases and calculates exact dates
Camera matching	<code>nodes/video_resource/</code>	Matches camera names and IPs (97 cameras)
Transform	<code>transform.py</code>	Converts the result into the backend API format
API	<code>api/</code>	FastAPI server exposing the pipeline
UI	<code>app.py, ui/</code>	Streamlit Search page and Data Explorer page
Evaluation	<code>evaluate/, annotate/</code>	Annotation tools and accuracy scripts
Demo / offline mode	<code>demo/</code>	Rule-based engine used when no API key is set

3. Technologies used

What	Tool
Language	Python
AI model	OpenAI GPT (any OpenAI-compatible model: Gemini, Groq, etc.)
Pipeline / workflow	LangGraph + LangChain
Backend API	FastAPI
Frontend / UI	Streamlit (Search page + Data Explorer page)
Validation	Pydantic, JSON Schema
Fuzzy matching	RapidFuzz
Charts / data	Altair, Pandas

Requirements

- **Software:** Python 3.10+, the packages in `requirements.txt`, a web browser, and optionally an OpenAI-compatible API key.
- **Hardware:** any normal laptop (4 GB+ RAM). No GPU needed, because the AI model runs on the provider's servers.

4. "How is it trained?"

It is not trained. No model was trained or fine-tuned in this project. Examiners ask this a lot, so answer it clearly:

"We use a **pre-trained Large Language Model** (like GPT) through its API. Instead of training, we use **prompt engineering**: detailed instructions, rules and examples in the prompt. We also use **function calling** to force the model to answer in a fixed JSON format. We didn't train a model because we had no large labelled dataset, and training is costly. Modern LLMs already understand English very well."

What the project *does* have is **evaluation**, meaning testing the model on labelled data:

- About 1,600 test queries (717 for events, 610 for fields, 308 for time).
- The model's answers were checked and corrected by a human using annotation tools ([annotate/](#)).
- Accuracy was measured with Precision, Recall and F1 score ([evaluate/](#)).

Results

Task	F1 score
Event selection	97.5%
Time understanding	93.8%
Field extraction	87.0%
Offline mode (no AI, rules only), event selection	87.1%

There's also an **offline rules mode** ([demo/](#)). If no API key is set, keyword and regex rules replace the LLM, so the demo still works. It also gives a good comparison: AI 97.5% versus rules 87.1%.

Dataset

- 2,000 synthetic (dummy) CCTV events over 60 days, generated by [scripts/generate_dummy_data.py](#), because real CCTV data is private.
- 15 analytics types (ANPR, Face Recognition, Crowd Detected, Safety Gear Violation, and more), each described by a JSON Schema in [artifacts/events_schema/](#).
- 97 cameras in [artifacts/video_resources.json](#).

5. Likely viva questions

Basics

1. What is NLP?

The field of making computers understand human language.

2. What is an LLM?

A Large Language Model: a neural network trained on huge amounts of text that can understand and generate language (e.g. GPT).

3. What is prompt engineering?

Writing instructions and rules for the LLM so that it gives the output you want, without retraining it.

4. What is a hallucination?

When the AI confidently makes something up. We handle it by validating every output against a whitelist or schema.

5. What is fuzzy matching?

Matching strings that are *similar*, not identical, so typos still match.

6. What is an API / FastAPI?

A way for programs to talk to each other. FastAPI is a Python framework for building them.

7. Why Streamlit?

It builds a web UI in pure Python quickly, which suits a demo.

8. What is function calling?

A feature where the LLM must reply by filling a JSON structure we define, instead of free text.

Project-specific

9. Why not let the LLM do everything?

It makes mistakes with dates and invents names. Code is exact and testable.

10. Why is camera matching done without AI?

The camera list is large and changes often. Matching needs to be fast, free and exact.

11. What does "last week" versus "last 7 days" give?

Last week = the previous Monday to Sunday. Last 7 days = a rolling window ending now.

12. What if the LLM is down?

Every stage has a fallback. The time stage, for example, defaults to "today". The app never crashes.

13. What is LangGraph?

A library that runs AI steps as a graph (a flowchart), so independent steps can run in parallel.

14. What are precision, recall and F1?

Precision: of what we predicted, how much was correct. Recall: of what was correct, how much we found. F1: a balance of the two.

15. Where did the data come from?

2,000 synthetic events generated by a script, since real CCTV data is private.

16. Can it handle multiple things in one sentence?

Yes. "Stolen bike *and* angry person" gives both ANPR and Face Recognition.

17. What happens for an unrelated query like "show me the weather"?

No events and no attributes are found, so it is marked "irrelevant" and the pipeline stops early.

18. Why temperature 0?

It makes the LLM's answers consistent every time, which is needed for testing.

Scope and future work

19. Limitations?

Field recall is lower (77.5%), single-word camera names sometimes fail, there's no Hindi support yet, and it depends on a paid API for the best accuracy.

20. Future scope?

Hindi/Hinglish queries, voice input, fine-tuning a small open-source model, connecting to live CCTV systems, and unit tests.

21. Real-world use?

Police control rooms, smart cities, factories (safety-gear checks), traffic departments.

6. How to run the demo

```
pip install -r requirements.txt
streamlit run app.py
```

1. On the **Search** page, run the default query and show the matching events and the filters that were applied.
2. Open the **Data Explorer** page to show the dataset, charts and CSV export.
3. It works even without an API key, because it falls back to offline rules mode.

Tip: Be ready to open and explain `graph.py`, `nodes/time/resolver.py` and `nodes/events/prompt.py`.